

TP4 - Docker – Réseaux

Partie 1 – Les bases

1/

Commande : `ifconfig`

Cette commande affiche la liste des interfaces réseau disponibles sur la machine.

On constate la présence de plusieurs interfaces :

- **lo** : l'interface de loopback, utilisée pour le trafic interne (127.0.0.1).
- **eth0** : l'interface réseau principale connectée à Internet ou au réseau local.
- **docker0** : une interface virtuelle créée automatiquement par Docker, servant de pont entre les conteneurs et l'hôte.

Cela montre que Docker ajoute sa propre interface réseau pour gérer la communication entre les conteneurs et le système hôte.

```

tpintegrationcontinue-clairecausse on 10 dev [?] is 1.0-SNAPSHOT via 1
) ifconfig
status: inactive
utun0: flags=8051<UP,POINTOPOINT,RUNNING,MULTICAST> mtu 1500
    inet6 fe80::ae2:5510:6a67:4bc1%utun0 prefixlen 64 scopeid 0xf
    nd6 options=201<PERFORMNUD,DAD>
utun1: flags=8051<UP,POINTOPOINT,RUNNING,MULTICAST> mtu 1380
    inet6 fe80::1c51:75ce:d78d:2f27%utun1 prefixlen 64 scopeid 0x10
    nd6 options=201<PERFORMNUD,DAD>
utun2: flags=8051<UP,POINTOPOINT,RUNNING,MULTICAST> mtu 2000
    inet6 fe80::9b27:9c3e:234e:f99b%utun2 prefixlen 64 scopeid 0x11
    nd6 options=201<PERFORMNUD,DAD>
utun3: flags=8051<UP,POINTOPOINT,RUNNING,MULTICAST> mtu 1000
    inet6 fe80::ce81:b1c:bd2c:69e%utun3 prefixlen 64 scopeid 0x12
    nd6 options=201<PERFORMNUD,DAD>
utun4: flags=8051<UP,POINTOPOINT,RUNNING,MULTICAST> mtu 1380
    inet6 fe80::f755:4b70:901b:beeb%utun4 prefixlen 64 scopeid 0x13
    nd6 options=201<PERFORMNUD,DAD>
utun5: flags=8051<UP,POINTOPOINT,RUNNING,MULTICAST> mtu 1380
    inet6 fe80::3dd3:980f:27ef:5b1e%utun5 prefixlen 64 scopeid 0x14
    nd6 options=201<PERFORMNUD,DAD>
utun6: flags=8051<UP,POINTOPOINT,RUNNING,MULTICAST> mtu 1380
    inet6 fe80::530:8ffe:7702:db0d%utun6 prefixlen 64 scopeid 0x15
    nd6 options=201<PERFORMNUD,DAD>
utun7: flags=8051<UP,POINTOPOINT,RUNNING,MULTICAST> mtu 1380
    inet6 fe80::d18c:eb4e:1365:362f%utun7 prefixlen 64 scopeid 0x16
    nd6 options=201<PERFORMNUD,DAD>
utun8: flags=8051<UP,POINTOPOINT,RUNNING,MULTICAST> mtu 1380
    inet6 fe80::1db0:b725:4811:1e7%utun8 prefixlen 64 scopeid 0x19
    nd6 options=201<PERFORMNUD,DAD>
utun9: flags=8051<UP,POINTOPOINT,RUNNING,MULTICAST> mtu 1380
    inet6 fe80::6046:cba3:81cd:7327%utun9 prefixlen 64 scopeid 0x1a
    nd6 options=201<PERFORMNUD,DAD>
(base)
tpintegrationcontinue-clairecausse on 10 dev [?] is 1.0-SNAPSHOT via 1

```

2/

J'avais encore en actif le conteneur de l'image ubuntu :

Pour repartir d'une situation sans aucun conteneur (actif ou stoppé), tu peux exécuter :

Commande : `docker rm -f conteneur`

Cette commande arrête et supprime le conteneur **conteneur**.

En relançant ensuite `docker ps -a`, aucun conteneur n'apparaît, ce qui confirme que tu repars bien d'un environnement vide.

3/

Commande : `docker network ls`

```
tpintegrationcontinue-clairecausse on ⓘ dev [?] is 📦 v1.0-SNAPSHOT
• > docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
582229382e77        PAS-net            bridge             local
ab37c889c7ad        PMIS-net           bridge             local
c785a6033407        bridge            bridge             local
cd12cd4680f6        host              host               local
7de461fda6fe        none              null               local
4b1db0b20331        sae-s5a01-2024-sujet03_default bridge             local
(base)
tpintegrationcontinue-clairecausse on ⓘ dev [?] is 📦 v1.0-SNAPSHOT
```

Cette commande affiche la liste des réseaux Docker existants sur la machine.

On y trouve généralement trois réseaux par défaut :

- **bridge** : le réseau par défaut utilisé pour connecter les conteneurs entre eux et à l'hôte via NAT.
- **host** : permet à un conteneur d'utiliser directement la pile réseau de l'hôte.
- **none** : réseau isolé sans connectivité externe.

Ces réseaux sont créés automatiquement par Docker à l'installation.

4/

Commande : `docker info`

```

tpintegrationcontinue-clairecausse on  dev [?+] is  v1.0-SNAPSHOT via 
> docker info
Default Runtime: runc
Init Binary: docker-init
containerd version: 05044ec0a9a75232cad458027ca83437aee3f4da
runc version: v1.2.5-0-g59923ef
init version: de40ad0
Security Options:
  seccomp
    Profile: builtin
  cgroupns
Kernel Version: 6.10.14-linuxkit
Operating System: Docker Desktop
OSType: linux
Architecture: aarch64
CPUs: 8
Total Memory: 7.654GiB
Name: docker-desktop
ID: c7c1870e-85e4-4a7c-8cdb-9ffbf2811e91
Docker Root Dir: /var/lib/docker
Debug Mode: false
HTTP Proxy: http.docker.internal:3128
HTTPS Proxy: http.docker.internal:3128
No Proxy: hubproxy.docker.internal
Labels:
  com.docker.desktop.address=unix:///Users/clairecausse/Library/Containers/com.docker.docker/Data/docker-cli.sock
Experimental: false
Insecure Registries:
  hubproxy.docker.internal:5555
  ::1/128
  127.0.0.0/8
Live Restore Enabled: false

```

Cette commande affiche la configuration complète de Docker, dont la liste des **drivers réseau disponibles**.

Dans la section **Network**, on observe généralement les drivers suivants :

- **bridge** : réseau par défaut, reliant les conteneurs via un pont virtuel et utilisant le NAT.
- **host** : donne au conteneur un accès direct au réseau de l'hôte.
- **overlay** : permet la communication entre conteneurs sur plusieurs hôtes (utilisé dans Docker Swarm).
- **macvlan** : attribue une adresse MAC unique à chaque conteneur pour qu'il apparaisse comme un appareil physique sur le réseau.
- **none** : désactive complètement le réseau du conteneur.

Ces drivers déterminent la façon dont les conteneurs communiquent entre eux et avec l'extérieur.

5/

Commande : `docker network inspect bridge | jq`

```
tpintegrationcontinue-clairecausse on  dev [?+] is  v1.0-SNAPSHOT via 
● > docker network inspect bridge | jq
[
  {
    "Name": "bridge",
    "Id": "c785a60334077c8c8e14d0f10ed6952f1cc76a72a055727fe3607f41c7b2e1f1",
    "Created": "2025-11-04T22:19:24.568847458Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "172.17.0.0/16",
          "Gateway": "172.17.0.1"
        }
      ]
    },
    "Internal": false
  }
]
```

Cette commande affiche toutes les informations détaillées du réseau **bridge**, mises en forme par `jq` pour une lecture plus claire.

Dans la section `"IPAM"`, on trouve le champ `"Config"` contenant la clé `"Subnet"`, par exemple : `"Subnet": "172.17.0.0/16"`

6/

Commande : `docker run -d --name www nginx`

Le conteneur `www` est lancé à partir de l'image `nginx` sans exposition de port vers l'extérieur.

7/

Commande : `docker inspect www | jq`

Cette commande affiche toutes les informations du conteneur `www`.

Dans la section `"NetworkSettings"`, on trouve la clé `"IPAddress"`, par exemple `"IPAddress": "172.17.0.2"`.

```
containers: {
  "1e14b697df77b9ecf4cea4bd4efba62f2af6b13": {
    "Name": "www",
    "EndpointID": "65f46f4db2863bcf485b7e2",
    "MacAddress": "86:09:70:c6:16:be",
    "IPv4Address": "172.17.0.2/16",
    "IPv6Address": ""
  }
}
```

Cela indique que le conteneur `www` tourne sur l'adresse IP interne `172.17.0.2`, correspondant à celle attribuée par le réseau bridge de Docker.

8/

Commande : `ping 172.17.0.2`

La commande `ping` permet de tester la connectivité entre la machine hôte (ou la VM) et le conteneur.

```
ping 172.17.0.2
PING 172.17.0.2 (172.17.0.2): 56 data bytes
Request timeout for icmp_seq 0
Request timeout for icmp_seq 1
Request timeout for icmp_seq 2
Request timeout for icmp_seq 3
```

Les réponses indiquent que le conteneur `www` est bien joignable sur le réseau bridge.

Commande : `curl 172.17.0.2`

Cette commande envoie une requête HTTP vers le serveur `nginx` tournant dans le conteneur.

La réponse renvoie le contenu HTML de la page d'accueil par défaut de `nginx`, confirmant que le service web est bien actif et accessible via son adresse IP interne.

9/

Commande : `curl 172.17.0.2`

Depuis la machine hôte, la requête vers l'adresse `172.17.0.2` échoue.

Cette adresse appartient au réseau interne bridge de Docker, isolé du système hôte.

Ainsi, le conteneur est accessible depuis d'autres conteneurs connectés au même réseau, mais **pas directement depuis l'hôte**, sauf si l'on publie

explicitement le port avec une option `-p` lors de la création du conteneur.

10/

Commande : `docker run --rm busybox ping -c 4 172.17.0.2`

```
tpintegrationcontinue-clairecausse on i? dev [?] is v1.0-SNAPSHOT via
> docker run --rm busybox ping -c 4 172.17.0.2
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
5bc51b87d4ec: Pull complete
Digest: sha256:e3652a00a2fabd16ce889f0aa32c38eec347b997e73bd09e69c962ec7f8732ee
Status: Downloaded newer image for busybox:latest
PING 172.17.0.2 (172.17.0.2): 56 data bytes
64 bytes from 172.17.0.2: seq=0 ttl=64 time=0.141 ms
64 bytes from 172.17.0.2: seq=1 ttl=64 time=0.162 ms
64 bytes from 172.17.0.2: seq=2 ttl=64 time=0.204 ms
64 bytes from 172.17.0.2: seq=3 ttl=64 time=0.233 ms

--- 172.17.0.2 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.141/0.185/0.233 ms
(base)
```

Docker télécharge l'image busybox si elle n'est pas encore présente, puis exécute un conteneur temporaire qui envoie quatre paquets ICMP vers l'adresse 172.17.0.2 du conteneur www.

Les réponses reçues confirment que le conteneur busybox peut communiquer avec le conteneur www à travers le réseau bridge.

Une fois le test terminé, le conteneur busybox est automatiquement supprimé grâce à l'option `--rm`.

11/

Commande : `docker run -it --rm busybox sh`

Le conteneur temporaire basé sur busybox est lancé en mode interactif.

```

tpintegrationcontinue-clairecausse on ? dev [?] is v1.0-SNAPSHOT via
) docker run -it --rm busybox sh
/ # ifconfig
eth0      Link encap:Ethernet  HWaddr F2:63:49:34:3E:7B
          inet addr:172.17.0.3  Bcast:172.17.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:65535  Metric:1
          RX packets:10 errors:0 dropped:0 overruns:0 frame:0
          TX packets:3 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:872 (872.0 B)  TX bytes:126 (126.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

```

À l'intérieur, la commande `ifconfig` permet de voir son adresse IP, généralement située dans la même plage que le réseau bridge, par exemple 172.17.0.3.

En exécutant `ping 172.17.0.2`, le conteneur busybox parvient à joindre le conteneur `www`.

Cela montre que les deux conteneurs peuvent communiquer entre eux car ils partagent le même réseau bridge par défaut.

```

RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

/ # ping 172.17.0.2
PING 172.17.0.2 (172.17.0.2): 56 data bytes
64 bytes from 172.17.0.2: seq=0 ttl=64 time=0.306 ms
64 bytes from 172.17.0.2: seq=1 ttl=64 time=0.242 ms
64 bytes from 172.17.0.2: seq=2 ttl=64 time=0.244 ms
64 bytes from 172.17.0.2: seq=3 ttl=64 time=0.291 ms
64 bytes from 172.17.0.2: seq=4 ttl=64 time=0.254 ms
64 bytes from 172.17.0.2: seq=5 ttl=64 time=0.271 ms
64 bytes from 172.17.0.2: seq=6 ttl=64 time=0.216 ms
64 bytes from 172.17.0.2: seq=7 ttl=64 time=0.232 ms
64 bytes from 172.17.0.2: seq=8 ttl=64 time=0.235 ms

```

12/

Commande : `docker run --rm busybox ping www`

Le ping échoue, car le conteneur busybox n'est pas dans le même réseau que le conteneur www.

```
(base)
tpintegrationcontinue-clairecausse on 1 dev [?] is v1.0-SNAPSHOT via
) docker run --rm busybox ping www
ping: bad address 'www'
(base)
tpintegrationcontinue-clairecausse on 1 dev [?] is v1.0-SNAPSHOT via
```

Par défaut, les conteneurs créés séparément ne peuvent pas résoudre leurs noms DNS respectifs sur le réseau bridge par défaut.

Le nom du conteneur www n'est donc pas connu dans ce contexte.

Pour que la résolution fonctionne, il faut les connecter à un réseau utilisateur créé manuellement avec `docker network create`, où Docker active la résolution de noms automatique entre conteneurs.

```
tpintegrationcontinue-clairecausse on 1 dev [?] is v1.0-SNAPSHOT via
) docker network create www
3aa492640492bbbe644861faffa851f4b97ae906bc384a851abc90aa08c9a2f2
(base)
tpintegrationcontinue-clairecausse on 1 dev [?] is v1.0-SNAPSHOT via
```

Partie 2 – Le réseau bridge

1/

Commande :

```
docker network create --driver bridge --subnet 192.168.50.0/24 --ip-range 192.168.50.96/27 --gateway 192.168.50.127 new_bridge
```

Cette commande crée un nouveau réseau nommé new_bridge avec le driver bridge.

```
tpintegrationcontinue-clairecausse on 1 dev [?] is v1.0-SNAPSHOT via
) docker network create --driver bridge --subnet 192.168.50.0/24 --ip-range 192.168.50.96/27 --gateway 192.168.50.127 new_bridge
844d4b43bad2e642dc37c08888b20487d9dd56fe07e41c5d224e8c753d3b087a
(base)
tpintegrationcontinue-clairecausse on 1 dev [?] is v1.0-SNAPSHOT via
```

Le sous-réseau complet est 192.168.50.0/24, la passerelle assignée est 192.168.50.127, et l'intervalle d'adresses disponible pour les conteneurs est compris entre 192.168.50.96 et 192.168.50.127.

2/

Commande : `docker network inspect new_bridge | jq`

Cette commande affiche la configuration complète du réseau new_bridge, mise en forme pour une meilleure lecture.

```
"Config": [
  {
    "Subnet": "192.168.50.0/24",
    "IPRange": "192.168.50.96/27",
    "Gateway": "192.168.50.127"
  }
]
```

Dans la section "IPAM" puis "Config", on retrouve les valeurs demandées :

- `"Subnet": "192.168.50.0/24"`
- `"IPRange": "192.168.50.96/27"`
- `"Gateway": "192.168.50.127"`

Ces informations confirment que le réseau a bien été créé avec la configuration spécifiée.

3/

Commande : `docker run -d --rm --name temp_www --network new_bridge nginx`

Un conteneur temporaire nommé temp_www est créé à partir de l'image nginx et relié au réseau new_bridge.

Il obtient automatiquement une adresse IP dans la plage 192.168.50.96/27, conformément à la configuration du réseau.

Le conteneur sera supprimé automatiquement à son arrêt grâce à l'option `--rm`.

4/

Commande : `docker run --rm --network new_bridge busybox ping temp_www`

Le conteneur busybox rejoint le même réseau new_bridge que temp_www.

```
tpintegrationcontinue-clairecausse on 🐧 dev [?] is 📦 v1.0-SNAPSHOT via 📡
○ ) docker run --rm --network new_bridge busybox ping temp_www
PING temp_www (192.168.50.96): 56 data bytes
64 bytes from 192.168.50.96: seq=0 ttl=64 time=0.119 ms
64 bytes from 192.168.50.96: seq=1 ttl=64 time=0.659 ms
64 bytes from 192.168.50.96: seq=2 ttl=64 time=0.148 ms
```

Le ping réussit, car sur un réseau Docker personnalisé, la **résolution DNS interne** est activée automatiquement.

Ainsi, les conteneurs peuvent se joindre en utilisant **leurs noms**, sans connaître leurs adresses IP.

5/

Commande : `docker inspect temp_www | jq`

Dans la section "NetworkSettings", on trouve `"IPAddress": "192.168.50.96"` (ou une valeur proche selon l'attribution). C'est l'adresse IP du conteneur temp_www sur le réseau new_bridge.

```
"Gateway": "192.168.50.127",
"IPAddress": "192.168.50.96",
"IPPrefixLen": 24,
```

En lançant `curl 192.168.50.96` depuis la **VM**, on obtient une réponse HTML de la page par défaut de nginx, confirmant que le service fonctionne.

Depuis la **machine hôte**, en revanche, `curl 192.168.50.96` échoue.

Le réseau new_bridge est un réseau interne Docker, non routé vers l'extérieur, donc inaccessible directement depuis l'hôte.

6/

Commande : `docker run -d --rm --name temp2_www --network new_bridge -p 8080:80 nginx`

Le conteneur temp2_www est créé sur le même réseau new_bridge, mais son port 80 interne est exposé sur le port 8080 de la machine hôte.

Depuis la machine hôte, l'exécution de `curl http://localhost:8080` renvoie le contenu HTML de la page par défaut de nginx.

```
tpintegrationcontinue-clairecausse on 12 dev [?] is v1.0-SNAPSHOT via 4
● curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
(base)
tpintegrationcontinue-clairecausse on 12 dev [?] is v1.0-SNAPSHOT via 4
```

Cela confirme que le port 80 du conteneur est bien accessible depuis l'extérieur via le port 8080 de l'hôte.

7/

Commande : `docker inspect temp2_www`

Cette commande affiche toutes les informations détaillées du conteneur `temp2_www`, notamment la section **"Ports"** qui indique que le port **80** du conteneur est **redirigé vers le port 8080** de la machine hôte (`"80/tcp": [{"HostPort": "8080"}]`).

```

tpintegrationcontinue-clairecausse on 🐧 dev [?:] is
> docker inspect temp2_www
    },
    "StopSignal": "SIGQUIT"
  },
  "NetworkSettings": {
    "Bridge": "",
    "SandboxID": "2b2951eed84180530a22467b36",
    "SandboxKey": "/var/run/docker/netns/2b2",
    "Ports": {
      "80/tcp": [
        {
          "HostIp": "0.0.0.0",
          "HostPort": "8080"
        },
        {
          "HostIp": "::",
          "HostPort": "8080"
        }
      ]
    }
  }
}

```

Commande : `docker ps`

Elle permet de **vérifier les processus actifs** et confirme que le conteneur `temp2_www` expose bien le **port 80 → 8080** dans la colonne **PORTS**.

```

tpintegrationcontinue-clairecausse on 🐧 dev [?:] is 📦 v1.0-SNAPSHOT via 🔍
● > docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
f4efde67e714   nginx    "/docker-entrypoint..." 2 minutes ago  Up 2 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  temp2_www
42eb289d7503   nginx    "/docker-entrypoint..." 4 minutes ago  Up 4 minutes  80/tcp                               temp_www
1e14b697df77   nginx    "/docker-entrypoint..." 45 minutes ago Up 45 minutes  80/tcp                               www
(base)
tpintegrationcontinue-clairecausse on 🐧 dev [?:] is 📦 v1.0-SNAPSHOT via 🔍

```

8/

Commande : `docker run --rm -it busybox ping temp2_www`

Cette commande crée un **conteneur temporaire** (grâce à `--rm`, il sera supprimé à l'arrêt) basé sur **busybox**, en **mode interactif** (`-it`).

Comme aucun réseau n'est précisé, il utilise le **réseau bridge par défaut**, et le **ping échoue** puisque le conteneur n'est **pas sur le même réseau** que `temp2_www` (qui est sur `new_bridge`).

9/

Commande: `ifconfig`

```
tpintegrationcontinue-clairecausse on ♀ dev [?] is 📦 v1.0-SNAPSHOT via 📡
> ifconfig
    status: inactive
en2: flags=8963<UP,BROADCAST,SMART,RUNNING,PROMISC,SIMPLEX,MULTICAST> mtu 1500
    options=460<TS04,TS06,CHANNEL_IO>
    ether 36:43:09:91:83:84
    media: autoselect <full-duplex>
    status: inactive
bridge0: flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST> mtu 1500
    options=63<RXCSUM,TXCSUM,TS04,TS06>
    ether 36:43:09:91:83:80
    Configuration:
        id 0:0:0:0:0:0 priority 0 hellotime 0 fwddelay 0
        maxage 0 holdcnt 0 proto stp maxaddr 100 timeout 1200
        root id 0:0:0:0:0:0 priority 0 ifcost 0 port 0
        ipfilter disabled flags 0x0
    member: en1 flags=3<LEARNING,DISCOVER>
        ifmaxaddr 0 port 8 priority 0 path cost 0
    member: en2 flags=3<LEARNING,DISCOVER>
        ifmaxaddr 0 port 9 priority 0 path cost 0
    nd6 options=201<PERFORMNUD,DAD>
    media: <unknown type>
    status: inactive
ap1: flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST> mtu 1500
    options=6460<TS04,TS06,CHANNEL_IO,PARTIAL_CSUM,ZEROINVERT_CSUM>
    ether b6:b3:c6:78:74:2b
```

Affiche les interfaces réseau de la VM, où l'on voit celle associée à **new_bridge**.

Commande: `docker rm -f $(docker ps -q --filter network=new_bridge)`

Supprime tous les conteneurs utilisant le réseau **new_bridge**.

```
tpintegrationcontinue-clairecausse on ♀ dev [?] is 📦 v1.0-SNAPS
● > docker rm -f $(docker ps -q --filter network=new_bridge)
f4efde67e714
42eb289d7503
(base)
```

Commande: `ifconfig`

En la relançant, on constate que **l'interface liée à new_bridge n'apparaît plus**, signe qu'elle est **désactivée** car plus aucun conteneur n'y est connecté.

10/

Commande: `docker run -d --name www --network new_bridge --ip 192.168.50.55 -p 8080:80 nginx`

Cette commande crée un conteneur nommé **www** basé sur **nginx**, connecté au réseau **new_bridge** avec l'adresse IP fixe **192.168.50.55**, et redirige le **port 80 du conteneur vers le port 8080** de la machine hôte.

11/

Commande: `docker network inspect new_bridge`

Cette commande affiche les informations détaillées du réseau **new_bridge**, notamment la section **"Containers"**, où l'on peut vérifier que le conteneur **www** est bien présent avec l'adresse IP **192.168.50.55**.

```
"ConfigOnly": false,
"Containers": {
  "8dae14e6a78ed59b477a7c97a6e61f6d2fccc3894b7522abfd1c5da76dc157ea": {
    "Name": "www",
    "EndpointID": "d8d66a3ca65ab6b25dde128b82f1fa7c8db5cc728baed8591f4b05e6fe18ec76",
    "MacAddress": "a6:82:a9:a3:b9:4f",
```

12/

Commande: `curl localhost:8080`

```
(base)
tpintegrationcontinue-clairecausse on 1 dev [?] is v1.0-SNAPSHOT via
> curl localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
(base)
tpintegrationcontinue-clairecausse on 1 dev [?] is v1.0-SNAPSHOT via
```

Cette commande envoie une requête HTTP vers le port **8080** de la VM, redirigé vers le **port 80** du conteneur **www**. La réponse affiche la **page d'accueil par défaut de Nginx**, prouvant que la redirection fonctionne correctement.

13/

Commande: `docker network rm new_bridge`

```
(base)
tpintegrationcontinue-clairecausse
> docker network rm new_bridge
new_bridge
(base)
tpintegrationcontinue-clairecausse
```

Supprime définitivement le réseau **new_bridge** (il faut qu'aucun conteneur n'y soit connecté).

Commande: `docker network ls`

Permet de vérifier la suppression : le réseau **new_bridge** ne figure plus dans la liste.

```
tpintegrationcontinue-clairecausse on 🐚 dev [?] is 📦 v1.0-SNAPSHOT
> docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
582229382e77        PAS-net            bridge              local
ab37c889c7ad        PMIS-net           bridge              local
6aa10b6fa7e8        bridge             bridge              local
cd12cd4680f6        host               host                local
7de461fda6fe        none               null                local
4b1db0b20331        sae-s5a01-2024-sujet03_default bridge              local
3aa492640492        www                bridge              local
(base)
```

14/

Commande: `docker network create --driver bridge --subnet 192.168.50.0/24 --ip-range 192.168.50.96/27 --gateway 192.168.50.127 -o com.docker.network.bridge.name=bridge0 new_bridge`

Crée le réseau **new_bridge** avec les mêmes paramètres qu'avant, mais en forçant le nom **bridge0** pour l'interface réseau.

Commande: `ifconfig`

Permet de constater qu'une nouvelle interface **bridge0** a bien été créée sur la VM.

15/

Commande: `docker run -d --name www --network new_bridge nginx`

Crée un conteneur **www** basé sur **nginx**, connecté au réseau **new_bridge** (interface **bridge0**), sans redirection de port ni adresse IP fixe.

16/

Commande: `docker network connect bridge www`

```
tpintegrationcontinue-clairecausse on  
● > docker network connect bridge www  
(base)
```

Connecte le conteneur **www** déjà présent sur **new_bridge** au réseau **bridge**, lui permettant d'avoir accès à deux réseaux simultanément.

17/

Commande: `docker network disconnect new_bridge www`

Déconnecte le conteneur **www** du réseau **new_bridge** tout en le laissant sur le réseau **bridge**.

Commande: `docker inspect www`

Permet de vérifier qu'il ne reste plus qu'une seule interface réseau, celle du réseau **bridge**.

Commande: `curl [IP_new_bridge]`

Ne répond plus (le conteneur n'est plus accessible via cette adresse).

Commande: `curl [IP_bridge]`

Renvoie toujours la page d'accueil **Nginx**, preuve que le conteneur fonctionne encore sur **bridge**.

Partie 3 – Le réseau host

1/

Listez les réseaux et identifiez le réseau host

Commande: `docker network ls`

```
tpintegrationcontinue-clairecausse on 1? dev [?] is v1.0-SNAPSHOT via
• ) docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
582229382e77        PAS-net            bridge              local
ab37c889c7ad        PMIS-net           bridge              local
6aa10b6fa7e8        bridge            bridge              local
cd12cd4680f6        host              host                local
0f214a392f64        new_bridge         bridge              local
7de461fda6fe        none              null                local
4b1db0b20331        sae-s5a01-2024-sujet03_default bridge              local
3aa492640492        www               bridge              local
(base)
```

Liste tous les réseaux Docker. On y trouve notamment le réseau host, qui partage directement l'interface réseau de la machine hôte sans passer par un pont virtuel.

2/

Créez un conteneur (temporaire, interactif) basé sur busybox dans le réseau bridge (par défaut)

Commande: `docker run -it --rm --network bridge busybox sh`

Lance un conteneur busybox temporaire sur le réseau bridge.

Dans le conteneur, commande: `ifconfig`

→ Affiche une interface eth0 avec une adresse IP isolée du réseau bridge.

```
(base)
tpintegrationcontinue-clairecausse on 1? dev [?] is v1.0-SNAPSHOT via
○ ) docker run -it --rm --network bridge busybox sh
/ # ifconfig
eth0      Link encap:Ethernet  HWaddr 6E:88:30:8F:FA:59
          inet addr:172.17.0.3  Bcast:172.17.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:65535  Metric:1
          RX packets:10 errors:0 dropped:0 overruns:0 frame:0
          TX packets:3 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:872 (872.0 B)  TX bytes:126 (126.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

/ #
```

3/

Créez un second conteneur (temporaire, interactif) basé sur busybox dans le réseau host

Commande: `docker run -it --rm --network host busybox sh`

Lance un autre conteneur busybox connecté au réseau host.

Dans le conteneur, commande: `ifconfig`

→ Les interfaces visibles sont identiques à celles de la machine hôte, car le conteneur partage son réseau.

```
tpintegrationcontinue-clairecausse on ♀ dev [?] is 📦 v1.0-SNAPSHOT via 🔥
) docker run -it --rm --network host busybox sh
    RX bytes:1364 (1.3 KiB)  TX bytes:1822 (1.7 KiB)

eth0      Link encap:Ethernet  HWaddr 02:9F:D3:79:6C:55
          inet addr:192.168.65.3  Bcast:192.168.65.255  Mask:255.255.255.0
          inet6 addr: fdc4:f303:9324::3/64 Scope:Global
          inet6 addr: fe80::9f:d3ff:fe79:6c55/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:65535  Metric:1
          RX packets:27104 errors:0 dropped:0 overruns:0 frame:0
          TX packets:25791 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:4234353 (4.0 MiB)  TX bytes:8719616 (8.3 MiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:2 errors:0 dropped:0 overruns:0 frame:0
          TX packets:2 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:140 (140.0 B)  TX bytes:140 (140.0 B)

services1 Link encap:Ethernet  HWaddr E2:CA:1E:E4:54:09
          inet addr:192.168.65.6  Bcast:0.0.0.0  Mask:255.255.255.255
          inet6 addr: fe80::e0ca:1eff:fee4:5409/64 Scope:Link
          inet6 addr: fdc4:f303:9324::6/128 Scope:Global
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:22771 errors:0 dropped:0 overruns:0 frame:0
          TX packets:27035 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
```

4/

Créez un réseau bridge br0 dans un second terminal et observez

Commande: `docker network create -d bridge br0`

Crée un réseau br0. Sur la machine hôte, commande: `ifconfig`

→ Une nouvelle interface br0 apparaît.

Retournez dans le premier conteneur (sur le réseau bridge) et tapez `ifconfig`

→ Aucune nouvelle interface n'apparaît, car le conteneur ne voit pas les réseaux créés après son démarrage.

Commande: `docker network rm br0`

```
tpintegrationcontinue-clairecausse on 🐧 dev [?!] is 📦 v1.0-SNAPSHOT V
⊗ ) docker run -it --rm --network bridge busybox sh
/ # ifconfig
eth0      Link encap:Ethernet  HWaddr D2:23:0B:DA:7F:76
          inet addr:172.17.0.3  Bcast:172.17.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:65535  Metric:1
          RX packets:10 errors:0 dropped:0 overruns:0 frame:0
          TX packets:3 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:872 (872.0 B)  TX bytes:126 (126.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

→ Supprime le réseau br0.

5/

Test avec un serveur web dans le réseau host

Dans le navigateur de la machine hôte, vérifiez qu'il n'y a aucun accès web actif.

Commande: `docker run --rm --network host --name www nginx`

Lance un conteneur www basé sur nginx dans le réseau host, sans exposer de port.

→ En accédant à <http://localhost> dans le navigateur, la page d'accueil Nginx s'affiche, car le conteneur utilise directement le port 80 de la VM.

```
tpintegrationcontinue-clairecausse on  dev [?i] is  v1.0-SNAPSHOT via 
○ docker run --rm --network host --name www nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2025/11/12 14:31:55 [notice] 1#1: using the "epoll" event method
2025/11/12 14:31:55 [notice] 1#1: nginx/1.29.3
2025/11/12 14:31:55 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2025/11/12 14:31:55 [notice] 1#1: OS: Linux 6.10.14-linuxkit
2025/11/12 14:31:55 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2025/11/12 14:31:55 [notice] 1#1: start worker processes
2025/11/12 14:31:55 [notice] 1#1: start worker process 30
2025/11/12 14:31:55 [notice] 1#1: start worker process 31
2025/11/12 14:31:55 [notice] 1#1: start worker process 32
2025/11/12 14:31:55 [notice] 1#1: start worker process 33
2025/11/12 14:31:55 [notice] 1#1: start worker process 34
2025/11/12 14:31:55 [notice] 1#1: start worker process 35
2025/11/12 14:31:55 [notice] 1#1: start worker process 36
2025/11/12 14:31:55 [notice] 1#1: start worker process 37
```